RAG 项目模块梳理与完善路线

当前项目可以理解成 5 个核心模块，加上 Web、配置、测试这些辅助模块。整体目标是完成“上传或放入资料 -> 构建索引 -> 网页问答 -> 可选知识图谱增强”的知识库问答闭环。

1. app：入口和编排层

主要文件：

- app/main.py：命令行入口，目前包含 ingest、ask、web 三个命令。
- app/graph.py：LangGraph 工作流，负责把问题送入向量检索、图谱检索和答案生成流程。
- app/config.py：从 .env 读取 OpenAI Key、模型、数据目录、Chroma 目录、Neo4j 参数。
- app/web.py 和 app/web_static/：本地网页服务与前端页面。

当前 CLI 流程：python -m app.main ingest；python -m app.main ask "问题"；python -m app.main web。

LangGraph 工作流：问题 -> 向量检索 -> 图谱检索 -> 生成答案。

不完善点：graph.py 目前每次问答都会连接 Neo4j，如果 Neo4j 没启动，普通 RAG 问答也可能受影响；缺少更友好的错误处理、状态提示和异步任务管理；网页索引是同步请求，文档多时页面会等待较久。

2. rag：向量检索模块

- rag/loaders.py：加载原始文档。目前只支持 .md 和 .txt。
- rag/splitter.py：把文档切成适合检索的 chunk。
- rag/vector_store.py：创建 Chroma 向量库，调用 OpenAI Embeddings 写入和检索。
- rag/retriever.py：把检索结果格式化成带编号的上下文。

不完善点：当前不支持 PDF，但 data/raw 里已经有 数学之美_副本.pdf；没有去重机制，重复运行 ingest 可能重复写入；没有文件变更检测，无法只增量处理新增或修改的文件；没有保留 PDF 页码等更细粒度的来源信息。

3. kg：知识图谱模块

- kg/extractor.py：调用 LLM 从文本里抽取实体和关系。
- kg/neo4j_store.py：把实体关系写入 Neo4j，并根据问题搜索图谱上下文。

不完善点：实体抽取是逐 chunk 调用 LLM，速度慢且成本高；图谱搜索目前偏简单，主要是关键词匹配；没有实体合并、关系置信度、重复关系处理；Neo4j 不可用时缺少降级逻辑。

4. chains：提示词和生成模块

- chains/prompts.py：定义问答 prompt，要求只根据上下文回答。
- chains/answer_chain.py：调用 ChatOpenAI 生成最终答案。

不完善点：没有答案质量检查；没有引用格式校验；没有判断上下文是否足够；没有流式输出。

5. tests：测试模块

目前只有 tests/test_neo4j_store.py，测试了关系类型规范化函数。

不完善点：测试覆盖较少。RAG 加载、切分、向量写入、CLI、Web API、Graph 流程都还没有测试。

6. 其他文件

- docker-compose.yml：用于启动 Neo4j。
- data/raw/：存放知识库原始文档。
- requirements.txt：项目依赖。
- README.md：项目说明和快速开始。

推荐完善顺序

1. 先完善文档加载。当前最明显的问题是 PDF 不能加载，应该先让 rag/loaders.py 支持 PDF。
2. 给普通 RAG 加降级能力。Neo4j 没启动时，ask 和网页问答应该仍能走向量检索。
3. 处理重复索引。避免每次 ingest 都把同一批文档重复写进 Chroma。
4. 完善 Web 体验。增加上传文件、索引进度、错误提示、历史问答和上下文展开。
5. 优化知识图谱。做实体去重、关系合并、图谱搜索增强，再考虑图谱路径推理。
6. 补测试。至少补 loader、splitter、CLI、Web API、Neo4j 降级测试。

下一步建议

建议下一步先做 PDF 加载 + Neo4j 降级。这两个做好后，项目就能更稳定地完成“上传资料 -> 建索引 -> 网页问答”的基本闭环。